Guía de Trabajos Prácticos

Escuela de Educación Técnica Nro1 Vte. Lopez

2021



Año	**Quinto**
Espacio	**Aplicaciones de Electrónica Digital**
Edición	**2da**
Original	Esteban Lemos - 2015
2da edicion	Ariel Wels - 2021

Contents

Trabajo Práctico Nro. 1 3

Uso de puertos de entrada y salida. **3**
Para poder realizar el trabajo 3

Trabajo Práctico Nro. 2 4

Dado electrónico **4**
Para poder realizar el trabajo 4

Trabajo Práctico Nro. 3 5

El auto fantastico **5**

Trabajo Práctico Nro. 4 6

Uso de Display **6**
Para poder realizar el trabajo 6

Trabajo Práctico Nro. 5 7

Uso del TIMER **7**
Para poder realizar el trabajo 7

Trabajo Práctico Nro. 6 9

Multiplexado de displays. **9**

Trabajo Práctico Nro. 7 10

Trabajo Final **10**

Trabajo Práctico Nro. 1

Uso de puertos de entrada y salida.

Utilizando un Arduino Nano (atmega328p), 4 LEDs y 4 Pulsadores se pide:

1. Mediante el uso de un pulsador controle el estado de un LED de manera que si el pulsador elegido esta presionado entonces el LED elegido debe de estar encendido. De manera extensiva si el pulsador elegido NO esta presionado entonces el LED elegido debe de estar apagado.
2. Hacer extensivo el funcionamiento a los cuatro pulsadores, de manera que se controlen a los cuatro LEDs mediante los cuatro PULSADORES.

Para poder realizar el trabajo

Se debe de realizar el diagrama de flujo, Elegir y configurar correctamente aquellos puertos a los que se conectaran los LEDs y los PULSADORES.

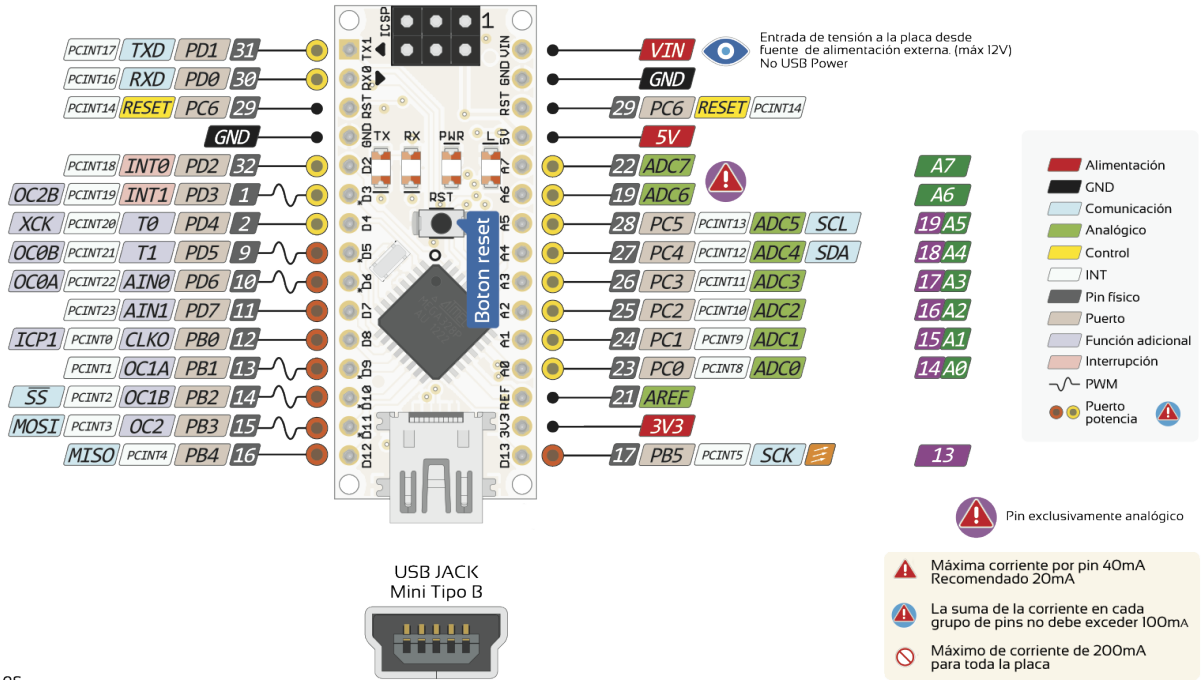


Figure 0.1: Pinout Arduino Nano

NOTA: Es importante tener en cuenta que los pulsadores se deben conectarse utilizando PULL UP o PULL DOWN externos[1] o bien configurar correctamente los PULL UP internos. Asi mismo los LEDs deben conectarse con su resistencia limitadora de corriente [2] asegurando no superar la corriente máxima que puede entregar el pin.

[1] valores usualmente de entre 4K7 y 10KΩ
[2] La resistencia limitadora debe ser como mínimo 330 Ω

Trabajo Práctico Nro. 2

Dado electrónico

1. Utilizando los pulsadores y LEDs para implementar un dado electrónico:

   (a) Al presionar el pulsador 1 se encienden los LEDs barajando las posibles respuestas rápidamente.

   (b) Al presionar el pulsador 2 se detiene el cambio de los LEDs, quedando encendidos aquellos que indican la "cara del dado que quedo hacia arriba".

   (c) Una vez que se indica el resultado, se debe poder presionar el pulsador 3 o 4 para apagar los LEDs de manera de volver a la condición inicial.

Para poder realizar el trabajo

Tome en cuenta que un dado tiene 6 posibles caras (1 a 6), lo que plantea diferentes posibilidades para controlar los LEDs.

Trabajo Práctico Nro. 3

El auto fantastico

1. Utilizar LEDs para realizar el efecto "auto fantástico: utilice 4 pulsadores para implementar las siguientes funciones:
   (a) Al presionar el pulsador 1 se enciende o se apaga el efecto.
   (b) Al presionar los pulsadores 2 y 3 simultáneamente **Baja** la velocidad del efecto.
   (c) Al presionar los pulsadores 2 y 4 simultáneamente **Sube** la velocidad del efecto.

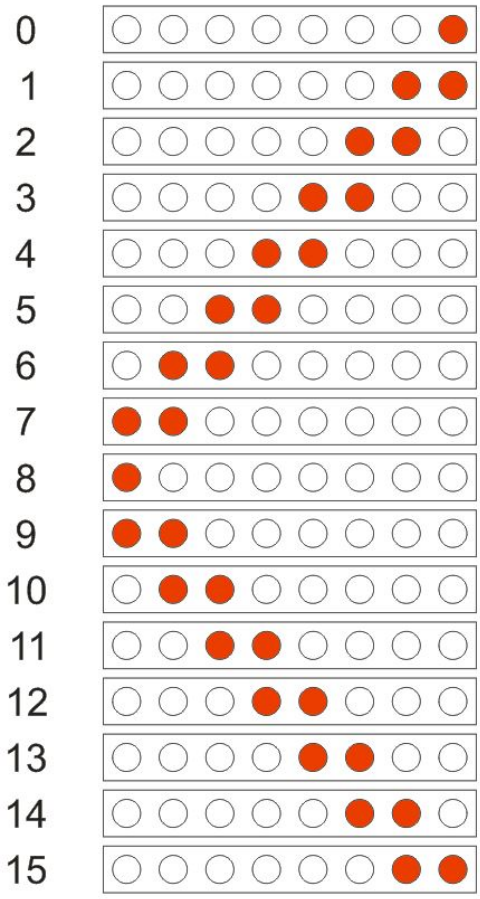


Figure 0.2: Ejemplo de efecto KITT con 8 leds

Trabajo Práctico Nro. 4

Uso de Display

Utilice un display de 7 segmentos para implementar lo siguiente:

1. Realizar un contador de 0 a 9 en binario que se incremente al presionar un pulsador. Mostrar el resultado de la cuenta en un display.
2. Agregar al problema anterior un pulsador que decremente el número.

   NOTA: el incremento o decremento debe de producirse en una unidad por pulsación.

Para poder realizar el trabajo

Tener en cuenta que los displays pueden ser anodo o catodo común, esto deberá tenerse en cuenta para conectar el común de los displays o bien a GND o bien a +Vcc para lograr el encendido.

Opcionalmente, se puede utilizar un decodificador BCD a 7 segmentos, CD4511(CMOS) o SN7447(TTL), esto simplifica la codificación pero aumenta la complejidad de el circuito.

Trabajo Práctico Nro. 5

Uso del TIMER

En el Trabajo Practico 3 (pagina 5) se realiza una secuencia y se elimina el rebote de los pulsadores mediante el uso de retardos. Esto produce 2 inconvenientes, por un lado un control de tiempo es inexacto e imposible de ajustar utilizando lenguajes de alto nivel. Por otro lado, eliminar los rebotes por espera y verificación implica el uso de código bloqueante, lo cual puede ser un inconveniente en muchas aplicaciones.

Retome el programa del TP3 y modifiquelo tal que:

1. Utilice Interrupciones por TIMER para controlar la velocidad del efecto de luces.
2. Utilizar detección de flanco y eliminación de rebote por interrupción eliminando el código bloqueante.

Para poder realizar el trabajo

Es necesario conocer la sintaxis especifica para el uso de interrupciones en cada plataforma. Para arduino, el uso de interrupciones de define de la siguiente forma:

```
#include <Arduino.h>


void setup() {
  // put your setup code here, to run once:

  ...


sei();  //Setea el bit en el registro que activa las interrupciones globales
//es decir, Habilita las interrupciones

//cli();  //Borra el bit en el registro que activa las interrupciones globales
//es decir, Deshabilita las interrupciones

}
void loop() {

    ...

}

ISR( Vector1 ) { //el argumento debe ser la fuente de interrupcion que se quiera
    utilizar

    ...

}

ISR( Vector2 ) { //el argumento debe ser la fuente de interrupcion que se quiera
    utilizar

    ...

}
```

Listing 1: codes/irq.c

La función ISR no debe declararse, solo es necesaria la definición y recibe como argumento el vector de la interrupción que se desea atender en esa definición.

Los vectores de interrupción existentes para cada microcontrolador están definidos en la hoja de datos, sin embargo, para el ATMEGA328p que utiliza en arduino NANO los vectores definidos son los siguientes:

```
// Vectores de interrupcion del ATMEGA328p (Arduino Nano):

INT0_vect          /*  External Interrupt Request 0  */
INT1_vect          /*  External Interrupt Request 1  */
PCINT0_vect        /*  Pin Change Interrupt Request 0  */
PCINT1_vect        /*  Pin Change Interrupt Request 0  */
PCINT2_vect        /*  Pin Change Interrupt Request 1  */
WDT_vect           /*  Watchdog Time-out Interrupt  */
TIMER2_COMPA_vect  /*  Timer/Counter2 Compare Match A  */
TIMER2_COMPB_vect  /*  Timer/Counter2 Compare Match A  */
TIMER2_OVF_vect    /*  Timer/Counter2 Overflow  */
TIMER1_CAPT_vect   /*  Timer/Counter1 Capture Event  */
TIMER1_COMPA_vect  /*  Timer/Counter1 Compare Match A  */
TIMER1_COMPB_vect  /*  Timer/Counter1 Compare Match B   */
TIMER1_OVF_vect    /*  Timer/Counter1 Overflow  */
TIMER0_COMPA_vect  /*  TimerCounter0 Compare Match A  */
TIMER0_COMPB_vect  /*  TimerCounter0 Compare Match B  */
TIMER0_OVF_vect    /*  Timer/Couner0 Overflow  */
SPI_STC_vect       /*  SPI Serial Transfer Complete  */
USART_RX_vect      /*  USART Rx Complete  */
USART_UDRE_vect    /*  USART, Data Register Empty  */
USART_TX_vect      /*  USART Tx Complete  */
ADC_vect           /*  ADC Conversion Complete  */
EE_READY_vect      /*  EEPROM Ready  */
ANALOG_COMP_vect   /*  Analog Comparator  */
TWI_vect           /*  Two-wire Serial Interface  */
SPM_READY_vect     /*  Store Program Memory Read  */
```

Listing 2: codes/irqvectors.c

Trabajo Práctico Nro. 6

Multiplexado de displays.

1. Mediante la interrupción del timer 0 realizar un programa que permita visualizar en dos de los displays los números 1 y 2 respectivamente. La frecuencia de multiplexado debe de ser de más de 20Hz.

2. Visualizar mediante la técnica anterior en los cuatro displays los números 1, 2, 3, 4.

Trabajo Práctico Nro. 7

Trabajo Final

1. Realizar un contador descendente de segundos que:

   (a) Mediante dos botones suba o baje el número a contar.
   (b) Un botón inicia la cuenta regresiva o la retoma en caso de haberse detenido.
   (c) Otro botón para la cuenta.
   (d) Al llegar a cero titilan los cuatro LEDs durante 5 segundos a una frecuencia de 1Hz.

   NOTAS:

   - No debe de haber rebotes.
   - El número máximo de segundos es 99.
   - El multiplexado se realiza con timer 0.
   - La eliminación de rebotes se realiza mediante detección de flancos.

References

[1] Werner Damm and David Harel. LSCs: Breathing life into message sequence charts. *Formal Methods in System Design*, 19(1):45–80, July 2001.

[2] Werner Damm and Bernd Westphal. Live and let die: LSC-based verification of UML-models. *Science of of Computer Programming*, 55(1–3):117–159, March 2005.

[3] Hartmut Ehrig and Martin Grosse-Rhode, editors. April 2002.

[4] David Harel and Rami Marelly. *Come, Let's Play: Scenario-Based Programming Using LSCs and the Play-Engine*. Springer-Verlag, 2003.

[5] ITU-T. *ITU-T Recommendation Z.120: Message Sequence Chart (MSC)*. ITU-T, 1999.

[6] Jochen Klose. *Live Sequence Charts: A Graphical Formalism for the Specification of Communication Behavior*. PhD thesis, Carl von Ossietzky Universität Oldenburg, 2003.

[7] Jochen Klose and Bernd Westphal. Relating LSC specifications to UML models. In Ehrig and Grosse-Rhode [3], pages 130–137.

[8] Sjouke Mauw and Victor Bos. Drawing Message Sequence Charts with LaTeX. *TUGBoat*, 22(1-2):87–92, March/June 2001.